```
    template <class T>
    Doub amotry(MatDoub_IO &p, VecDoub_O &y, VecDoub_IO &psum,
        const Int ihi, const Doub fac, T &func)
    Helper function: Extrapolates by a factor fac through the face of the simplex across from
    the high point, tries it, and replaces the high point if the new point is better.
    {
        VecDoub ptry(ndim);
        Doub fac1=(1.0-fac)/ndim;
        Doub fac2=fac1-fac;
        for (Int j=0;j<ndim;j++)
            ptry[j]=psum[j]*fac1-p[ihi][j]*fac2;
        Doub ytry=func(ptry);           Evaluate the function at the trial point.
        if (ytry < y[ihi]) {            If it's better than the highest, then replace the
            y[ihi]=ytry;                    highest.
            for (Int j=0;j<ndim;j++) {
                psum[j] += ptry[j]-p[ihi][j];
                p[ihi][j]=ptry[j];
            }
        }
        return ytry;
    }
};
```


CITED REFERENCES AND FURTHER READING:

Nelder, J.A., and Mead, R. 1965, "A Simplex Method for Function Minimization," *Computer Journal*, vol. 7, pp. 308–313.[1]

Yarbro, L.A., and Deming, S.N. 1974, "Selection and Preprocessing of Factors for Simplex Optimization," *Analytica Chimica Acta*, vol. 73, pp. 391–398.

Jacoby, S.L.S, Kowalik, J.S., and Pizzo, J.T. 1972, *Iterative Methods for Nonlinear Optimization Problems* (Englewood Cliffs, NJ: Prentice-Hall).

10.6 Line Methods in Multidimensions

We know (§10.2 – §10.4) how to minimize a function of one variable. If we start at a point $\mathbf{P}$ in N-dimensional space, and proceed from there in some vector direction $\mathbf{n}$, then any function of N variables $f(\mathbf{P})$ can be minimized along the line $\mathbf{n}$ by our one-dimensional methods. One can dream up various multidimensional minimization methods that consist of sequences of such line minimizations. Different methods will differ only by how, at each stage, they choose the next direction $\mathbf{n}$ to try. The minimization methods in the next two sections fall under this general schema of successive line minimizations. (The quasi-Newton algorithm in §10.9 does not need very accurate line minimizations. Accordingly, it uses the approximate line minimization routine, `lnsrch` from §9.7.1.)

In this section we provide the line minimization routine `linmin` as part of the base class `Linemethod` from which we will derive the minimization methods in the next two sections. These minimization routines regard `linmin` as a black-box subalgorithm, whose definition is

> `linmin`: Given as input the vectors $\mathbf{P}$ and $\mathbf{n}$, and the function f, find the scalar λ that minimizes $f(\mathbf{P} + \lambda\mathbf{n})$. Replace $\mathbf{P}$ by $\mathbf{P} + \lambda\mathbf{n}$. Replace $\mathbf{n}$ by $\lambda\mathbf{n}$. Done.

Since we will want to use `linmin` with methods whose choice of successive directions does not involve explicit computation of the function's gradient, `linmin` itself cannot use gradient information. Later, in §10.8, we will consider a method that does use gradient information. Accordingly, there we provide a routine `dlinmin` that makes use of this information to reduce the total computational burden.

The obvious way to implement `linmin` is to use the methods of one-dimensional minimization described in §10.2 – §10.4, but to rewrite the programs of those sections so that their bookkeeping is done on vector-valued points **P** (all lying along a given direction **n**) rather than scalar-valued abscissas x. That straightforward task produces long routines densely populated with "`for(k=0;k<n;k++)`" loops.

As an alternative, we can simply reuse the one-dimensional minimization routines by constructing a functor `F1dim`, which gives the value of your function, say `func`, along the line going through the point `p` in the direction `xi`. The function `linmin` calls our familiar one-dimensional routine `Brent` (§10.4) and instructs it to minimize `F1dim`. The routine `linmin` communicates with `F1dim` "over the head" of `Brent` through the constructor, our usual C++ idiom.

mins_ndim.h

```
template <class T>
struct Linemethod {
```

Base class for line-minimization algorithms. Provides the line-minimization routine `linmin`.

```
    VecDoub p;
    VecDoub xi;
    T &func;
    Int n;
    Linemethod(T &funcc) : func(funcc) {}
```

Constructor argument is the user-supplied function or functor to be minimized.

```
    Doub linmin()
```

Line-minimization routine. Given an `n`-dimensional point `p[0..n-1]` and an `n`-dimensional direction `xi[0..n-1]`, moves and resets `p` to where the function or functor `func(p)` takes on a minimum along the direction `xi` from `p`, and replaces `xi` by the actual vector displacement that `p` was moved. Also returns the value of `func` at the returned location `p`. This is actually all accomplished by calling the routines `bracket` and `minimize` of `Brent`.

```
    {
        Doub ax,xx,xmin;
        n=p.size();
        F1dim<T> f1dim(p,xi,func);
        ax=0.0;                                  Initial guess for brackets.
        xx=1.0;
        Brent brent;
        brent.bracket(ax,xx,f1dim);
        xmin=brent.minimize(f1dim);
        for (Int j=0;j<n;j++) {                  Construct the vector results to return.
            xi[j] *= xmin;
            p[j] += xi[j];
        }
        return brent.fmin;
    }
};
```

mins_ndim.h

```
template <class T>
struct F1dim {
```

Must accompany `linmin` in `Linemethod`.

```
    const VecDoub &p;
    const VecDoub &xi;
    Int n;
    T &func;
    VecDoub xt;
```